

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Screensaver de fuegos artificiales con OpenMP

Informe final del Proyecto #1

Semestre 2 · 2026

Adrián González · 23152

Fernando Mendoza · 19644

Hansel López · 19026

Repositorio: github.com/Ikeel04/Screensaver-PPD

Versión documentada: 53a2de3

Contenido

Estructura del informe y trazabilidad de los entregables

Resumen ejecutivo	03
1. Introducción y antecedentes	04
2. Diseño de la solución	05
3. Implementación	06
4. Metodología experimental	07
5. Resultados y discusión	08-09
6. Conclusiones y recomendaciones	10
Bibliografía	11
Anexo 1. Diagrama de flujo	12
Anexo 2. Catálogo de funciones	13-14
Anexo 3. Bitácora de pruebas	15-16

TRAZABILIDAD FRENTE A LA RÚBRICA

El informe documenta las versiones secuencial, paralela V1 y paralela V2; el uso de OpenMP; la parametrización defensiva; el cálculo de speedup y eficiencia; y la verificación de equivalencia por checksum. Los archivos fuente, scripts y resultados quedan en el repositorio junto con este PDF.

Resumen ejecutivo

Qué se construyó, cómo se paralelizó y qué se comprobó

RESULTADO

Se implementó un screensaver de fuegos artificiales en C++17 que actualiza partículas con gravedad, resistencia del aire, viento oscilante, rebotes, colores pseudoaleatorios y desvanecimiento. La escena se dibuja mediante OpenGL 3.3 con GLFW y GLEW. El programa recibe N y otros parámetros por línea de comandos, muestra FPS y permite cambiar entre tres implementaciones durante la ejecución.

PARALELIZACIÓN

La versión secuencial constituye la línea base. par1 aplica un parallel for por fase; par2 agrupa el frame en una única región paralela, usando omp for, reduction, single y nowait. El diseño sigue PCAM: partición por rangos de partículas, comunicación mínima, aglomeración contigua y mapeo estático.

EVIDENCIA

En $N = 500,000$, par1 llegó a $3.955\times$ de speedup con 16 hilos y par2 a $3.891\times$; todas las pruebas registraron checksum igual. La bitácora contiene 10 repeticiones por configuración y los CSV quedan en results/.

CUMPLIMIENTO

El repositorio contiene código C/C++ propio, OpenMP, versión secuencial y dos paralelas, validación defensiva, README, scripts reproducibles, mediciones de speedup/eficiencia, diagramas, catálogo de funciones y este informe PDF.

1. Introducción y antecedentes

Problema, motivación y objetivos del proyecto

CONTEXTO

OpenMP ofrece un modelo de programación paralela de memoria compartida para transformar iterativamente un programa secuencial. El proyecto aplica ese modelo a una escena dinámica donde cada partícula puede actualizarse de forma independiente, mientras el hilo principal conserva el control del contexto gráfico.

PROBLEMA

Un screensaver con muchos elementos debe conservar una animación fluida mientras calcula posiciones, velocidades, colores, vida y vértices. El costo crece con N , por lo que se busca reducir el tiempo de actualización sin alterar el resultado físico ni bloquear el renderizado.

OBJETIVO GENERAL

Diseñar e implementar un screensaver parametrizable con una línea base secuencial y versiones paralelas OpenMP, y cuantificar la mejora mediante speedup y eficiencia.

OBJETIVOS ESPECÍFICOS

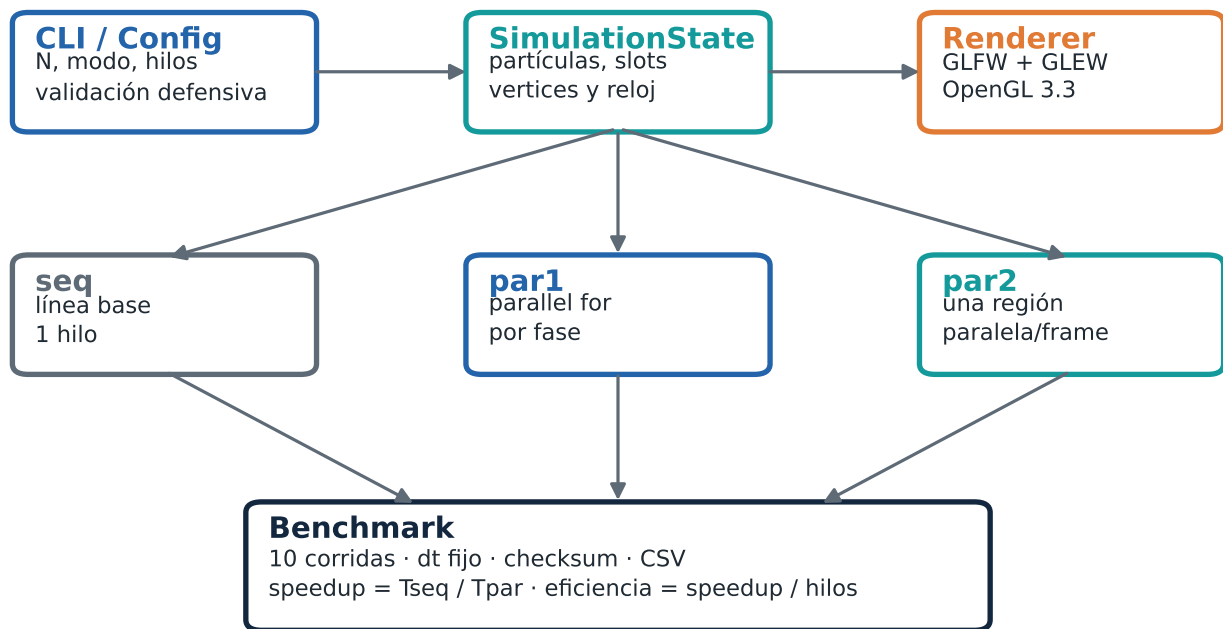
(1) Implementar la simulación física de partículas y su visualización. (2) Aplicar PCAM para identificar la unidad de trabajo. (3) Incorporar sincronización correcta y evitar carreras. (4) Comparar dos estrategias paralelas. (5) Construir una bitácora reproducible con al menos 10 mediciones por prueba.

MARCO TÉCNICO

OpenMP define directivas, cláusulas y un modelo de memoria para paralelismo en C/C++ [1]. GLFW abstrae ventana, contexto y eventos [2], mientras OpenGL provee la API de renderizado [3]. La combinación permite mantener la simulación en CPU y el dibujo en el hilo que posee el contexto gráfico.

2. Diseño de la solución

Separación entre configuración, simulación y renderizado



DECISIONES DE DISEÑO

- N partículas se divide en rangos contiguos asignados a E fuegos artificiales; cada partícula escribe únicamente su propio estado y vértice.
- La fase de lanzamientos se mantiene secuencial porque usa un generador aleatorio compartido y tiene costo $O(E)$, con $E \leq 64$.
- El renderizado permanece en el hilo principal: el contexto OpenGL no se comparte con las regiones de cálculo.

3. Implementación

Física, versiones de simulación, memoria compartida y render

MODELO FÍSICO

Cada partícula contiene posición, velocidad, color, vida, decaimiento y fase. En cada paso se aplica gravedad y arrastre; el viento depende del tiempo y la altura; el suelo y los límites laterales aplican restitución y fricción. La opacidad se deriva de la vida restante. Cuando un slot queda sin partículas vivas, se programa un nuevo fuego.

VERSIÓN SECUENCIAL

`stepSequential` procesa todas las partículas en un `for`, acumula partículas vivas por slot, decide relanzamientos y escribe los vértices. Es la referencia para `speedup` y `checksum`.

PAR1

`stepParallelV1` paraleliza la actualización con `reduction` sobre `aliveCounts` y abre `parallel for` adicionales para inicializar cada explosión. Es una mejora directa y didáctica, pero paga el costo de varias regiones paralelas por frame.

PAR2

`stepParallelV2` crea una región `parallel` por frame. El `omp for` actualiza y reduce; `omp single` ejecuta `scheduleLaunches` porque el RNG de slots es compartido; luego `omp for nowait` inicializa rangos independientes. Las barreras implícitas garantizan que la lista y los contadores estén listos antes de usarse.

PROGRAMACIÓN DEFENSIVA

`config.cpp` valida tipos, rangos, cantidades de hilos, tamaño mínimo de ventana, número de frames y opciones de modo. Los errores producen mensajes claros y salida controlada. `std::vector` administra memoria y `Renderer` libera GLFW/OpenGL mediante `RAII`.

4. Metodología experimental

Protocolo para medir rendimiento, corrección y experiencia visual

ENTORNO

Equipo de medición: AMD Ryzen AI 9 365, 10 núcleos / 20 hilos, x86_64, entorno Linux/WSL2. Compilador GCC 13.3.0, CMake 3.16+, OpenMP 4.5 reportado por GCC y build Release. La ventana interactiva requiere OpenGL 3.3, GLFW y GLEW; las mediciones de speedup se aislaron con `--sim-only` para no mezclar el costo del driver gráfico.

PROTOCOLO

- Se fijó $dt = 1/60$ s, semilla 2026 y el mismo estado inicial para seq, par1 y par2.
- Cada prueba ejecutó 30 frames de calentamiento y 100 frames medidos; se repitió 10 veces.
- N tomó los valores 10,000; 100,000; 500,000. Se probaron 4, 8 y 16 hilos en las corridas pequeñas y medianas; la corrida N=100,000 incluyó además 2 hilos.
- Se midieron por separado tiempo de simulación, tiempo total de frame y FPS teóricos; los resultados se escribieron en CSV.
- El checksum suma posiciones y vida de todas las partículas. Un resultado igual indica equivalencia dentro de la tolerancia configurada.

LIMITACIONES

Los FPS de esta bitácora son una métrica del loop sin renderizado y no sustituyen la observación visual del screensaver a 60 FPS. El comportamiento gráfico debe validarse en el equipo de presentación con VSync activo y luego con `--no-vsync` para aislar el límite de la simulación.

5. Resultados y discusión

Mediciones de simulación sin renderizado, 10 repeticiones por configuración

CONDICIÓN DE REFERENCIA

Las mediciones se ejecutaron en Release con $N = 10,000$, $100,000$ y $500,000$ partículas, 6 fuegos, $dt = 1/60$ s, 100 frames por corrida y 10 corridas por configuración. Se usó una semilla fija y se descartaron 30 frames de calentamiento. El tiempo reportado es el promedio del cálculo de simulación por frame; no incluye espera de VSync.

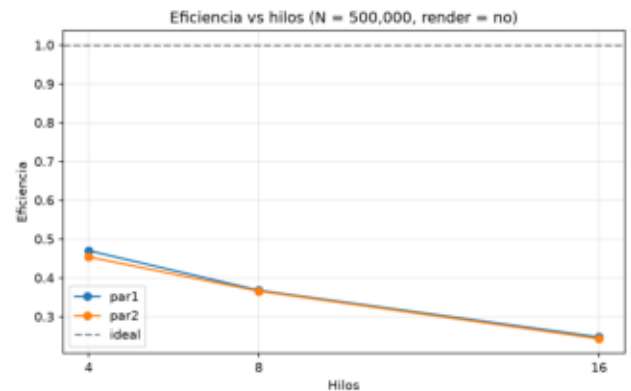
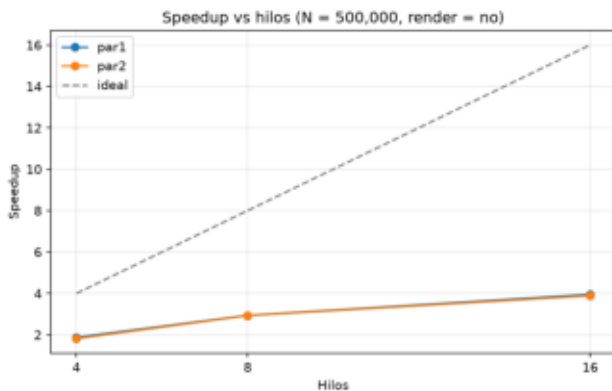
Modo	Hilos	ms/frame	σ	Speedup	Eficiencia	FPS	Checksum
seq	1	6.824	0.158	1.000	1.000	146.609	1
par1	4	3.629	0.044	1.880	0.470	275.568	1
par2	4	3.759	0.109	1.815	0.454	266.206	1
par1	8	2.320	0.085	2.942	0.368	431.592	1
par2	8	2.332	0.048	2.926	0.366	428.935	1
par1	16	1.725	0.040	3.955	0.247	579.770	1
par2	16	1.754	0.035	3.891	0.243	570.338	1

LECTURA DE RESULTADOS

- Con $N = 500,000$, par1 alcanza speedup $3.955\times$ con 16 hilos y par2 $3.891\times$; la eficiencia es 24.7% y 24.3% , respectivamente.
- Con $N = 10,000$ la carga es pequeña y domina el costo de coordinación: par2 llega a $2.634\times$ con 16 hilos, mientras par1 presenta mayor variabilidad.
- Todos los casos marcaron checksum igual. Esto comprueba que las versiones paralelas producen el mismo estado final bajo la semilla y el paso temporal controlados.
- La mejora no es lineal: memoria, barreras, fork/join y el límite de ancho de banda reducen la eficiencia al aumentar hilos. El resultado es consistente con el tamaño de la carga.

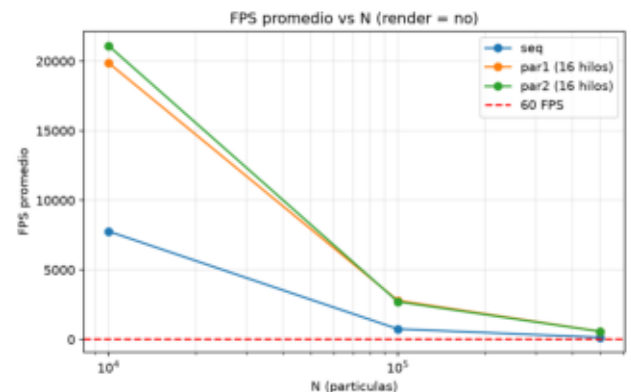
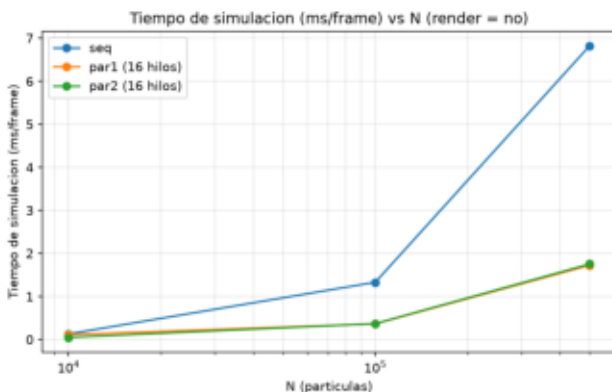
5. Resultados y discusión

Escalabilidad observada en la simulación



Speedup frente a hilos

Eficiencia frente a hilos



Tiempo por frame frente a N

FPS teóricos del modo sim-only

6. Conclusiones y recomendaciones

Evaluación final frente a los objetivos y requisitos

CONCLUSIONES

La solución cumple el flujo completo del proyecto: recibe N , genera una escena colorida con movimiento y física, muestra FPS, separa versión secuencial y paralelas, y usa OpenMP con mecanismos explícitos de reducción, sincronización y regiones críticas de control.

CONCLUSIONES

La paralelización produce una mejora medible. Para cargas grandes, 16 hilos redujo aproximadamente cuatro veces el tiempo de simulación frente a la línea base. La eficiencia menor que 1 es esperable: la coordinación, las barreras, el acceso a memoria y la parte secuencial limitan el escalamiento.

CONCLUSIONES

La igualdad de checksum en las pruebas da evidencia de corrección funcional entre modos. La arquitectura mantiene el renderizado fuera de OpenMP, evitando acceso concurrente al contexto gráfico.

RECOMENDACIONES

Para una evaluación en vivo, probar N creciente con VSync activo y registrar FPS observados; fijar afinidad de hilos si se requiere mayor estabilidad; repetir las mediciones en el equipo de evaluación; y conservar los CSV originales junto con la versión exacta del compilador.

ENTREGABLES

El PDF actual acompaña el código fuente C/C++, README, CMakeLists.txt, scripts de benchmark, CSV, logs y gráficas del directorio results/. No se incluyen ejecutables como parte del entregable documental.

Bibliografía

Fuentes técnicas y académicas consultadas

[1] OPENMP ARCHITECTURE REVIEW BOARD

OpenMP API Specification 5.2. Especificación del modelo de ejecución, memoria, directivas, reducciones y sincronización. <https://www.openmp.org/spec-html/5.2/openmp.html>

[2] GLFW PROJECT

GLFW 3.5 Documentation. API multiplataforma para ventanas, contextos OpenGL, entrada y eventos. <https://www.glfw.org/docs/latest/>

[3] KHRONOS GROUP

OpenGL Registry and Reference Pages. Especificaciones y referencia de la API OpenGL. <https://registry.khronos.org/OpenGL/>

[4] FOSTER, IAN

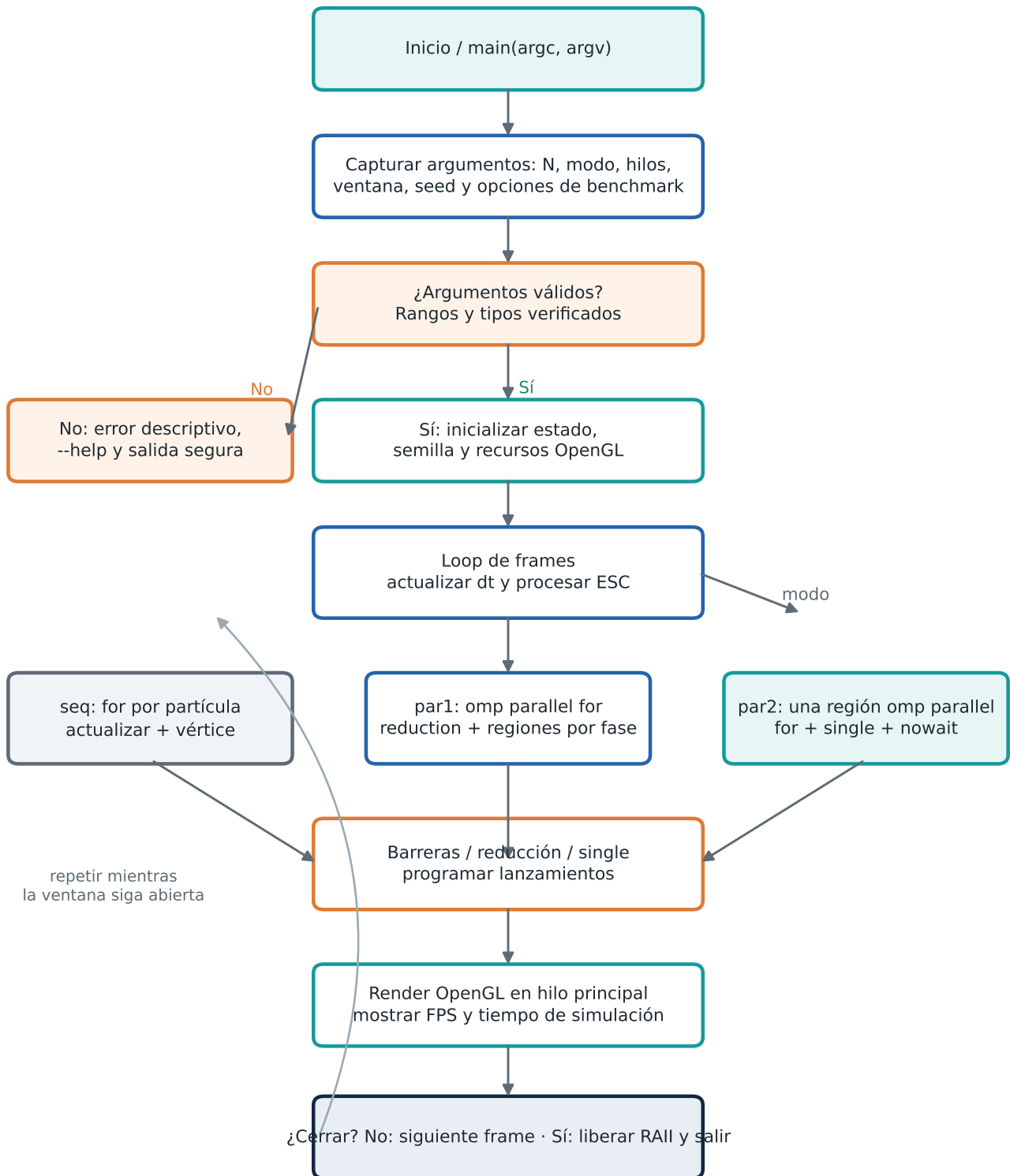
Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Addison-Wesley, 1995. Método PCAM: partición, comunicación, aglomeración y mapeo.

[5] UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida, Semestre 2, 2026. Proyecto #1: requisitos, contenido y rúbrica de evaluación.

Anexo 1. Diagrama de flujo

Flujo completo: argumentos, validación, simulación paralela y salida



Anexo 2. Catálogo de funciones

Entradas, salidas y propósito de las rutinas principales

parseArguments	argc, argv, Config&	Lee opciones, valida rangos y llena la configuración.
	Salida: ParseStatus; error por excepción	
initializeSimulation	State&, N, E, halfWidth, seed	Reserva vectores, reparte partículas por slots y prepara la semilla.
	Salida: void; estado inicializado	
stepSimulation	State&, SimulationMode, dt	Selecciona seq, par1 o par2 para procesar un frame.
	Salida: void; estado avanzado	
stepSequential	State&, dt	Línea base: actualiza partículas, cuenta vivas y relanza fuegos.
	Salida: void; estado avanzado	
stepParallelV1	State&, dt	Paraleliza cada fase con regiones parallel for independientes.
	Salida: void; estado avanzado	
stepParallelV2	State&, dt	Agrupar el frame en una región parallel con for, single y nowait.
	Salida: void; estado avanzado	
scheduleLaunches	State&, dt	Decide secuencialmente qué fuegos relanzar usando el RNG compartido.
	Salida: void; launchList actualizado	
Tipos compartidos: SimulationState agrupa partículas, vértices, slots, contadores y reloj; Renderer administra GLFW/OpenGL		

Anexo 2. Catálogo de funciones

Entradas, salidas y propósito de las rutinas principales

scheduleLaunches	State&, dt	Decide secuencialmente qué fuegos relanzar usando el RNG compartido.
	Salida: void; launchList actualizado	
updateParticle	Particle&, dt, time, drag, halfWidth	Aplica gravedad, viento, arrastre, rebotes, vida y desvanecimiento.
	Salida: bool; viva/apagada	
writeVertex	const Particle&, Vertex&, time	Convierte el estado físico a posición y color para OpenGL.
	Salida: void; vértice actualizado	
computeChecksum	const SimulationState&	Suma posiciones y vidas para verificar equivalencia entre versiones.
	Salida: double	
runScreensaver	const Config&	Crea ventana, ejecuta el loop interactivo, FPS y cambio de modo.
	Salida: int; código de salida	
runBenchmark	const Config&, Renderer*	Ejecuta 10 mediciones, calcula estadísticas y escribe CSV.
	Salida: int; código de salida	

Mecanismos de seguridad: validación de rangos, excepciones descriptivas, std::vector para memoria, checksum y destructores

Anexo 3. Bitácora de pruebas

Diez mediciones de una configuración representativa

CONFIGURACIÓN

N = 500,000; modo par2; 16 hilos; 6 fuegos; 100 frames por medición; 30 frames de calentamiento; semilla fija 2026; sin renderizado. La columna checksum confirma que el estado final coincide con la referencia secuencial.

Medición	Sim ms/frame	Frame ms	FPS	Checksum
1	1.7159	1.7162	582.7	igual
2	1.8404	1.8407	543.3	igual
3	1.7636	1.7639	566.9	igual
4	1.7269	1.7271	579.0	igual
5	1.7620	1.7622	567.5	igual
6	1.7790	1.7792	562.0	igual
7	1.7542	1.7545	570.0	igual
8	1.7292	1.7294	578.2	igual
9	1.7317	1.7319	577.4	igual
10	1.7348	1.7350	576.4	igual

CRITERIO DE CÁLCULO

- Tiempo promedio: media aritmética de los 10 valores de simulación por frame.
- Speedup: T_{seq} / T_{par} . Eficiencia: $speedup / \text{número de hilos}$.
- Los CSV completos y los logs de cada N quedan versionados en results/ para inspección y reproducción.

Anexo 3. Bitácora de pruebas

Resumen de todas las configuraciones agregadas al repositorio

COBERTURA

Se registraron 10 mediciones por combinación de N, modo y cantidad de hilos. La tabla resumen contiene 3 valores de N y las variantes seq, par1 y par2. Las columnas incluyen promedio, desviación estándar, máximo, speedup, eficiencia, FPS y checksum_ok.

ARCHIVOS DE EVIDENCIA

results/benchmark.csv conserva las mediciones individuales; results/benchmark_resumen.csv conserva los agregados; results/log_N10000.txt y results/log_N500000.txt conservan la salida de consola; y las imágenes *_simonly.png muestran las tendencias.

INTERPRETACIÓN

Los resultados deben leerse como una caracterización de la simulación en el equipo indicado. El modo par2 reduce regiones fork/join, pero no necesariamente supera a par1 en todos los tamaños: el costo de coordinación y el tamaño de la carga determinan el punto de equilibrio.

REPRODUCCIÓN

```
cmake -S . -B build && cmake --build build; ./build/screensaver 500000 --benchmark --sim-only  
--runs 10 --frames 100 --thread-list 4,8,16 --csv results/benchmark.csv; python3  
scripts/analyze_results.py
```